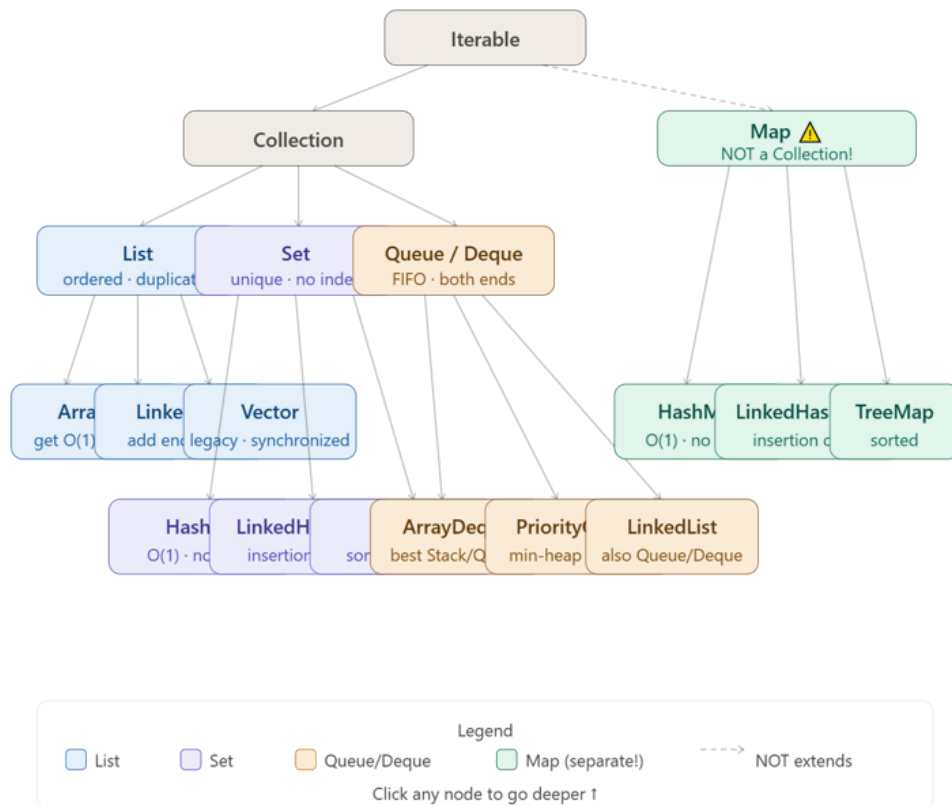
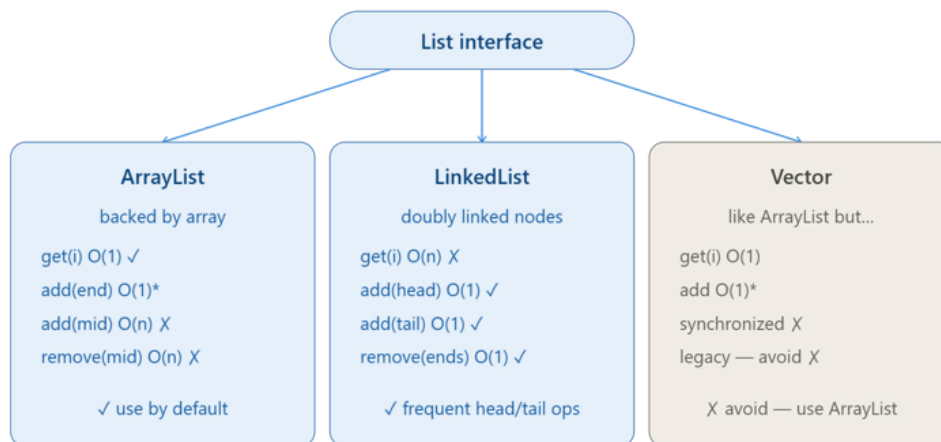


Collection

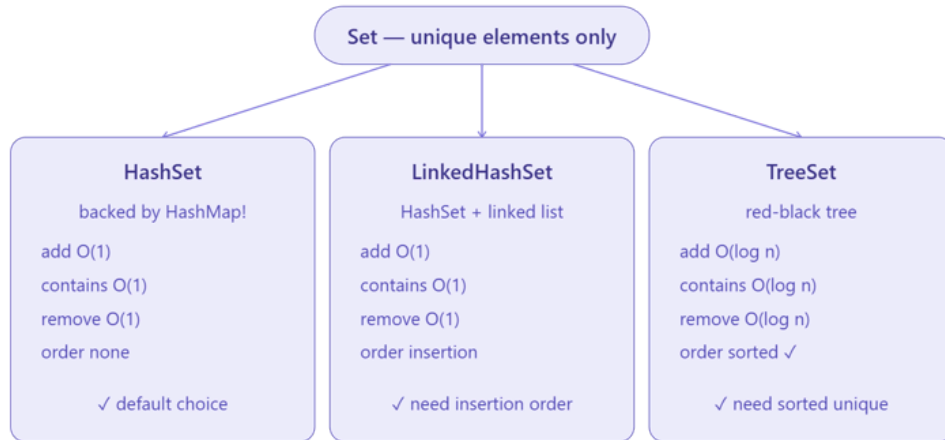


Full tree List Set Map Queue

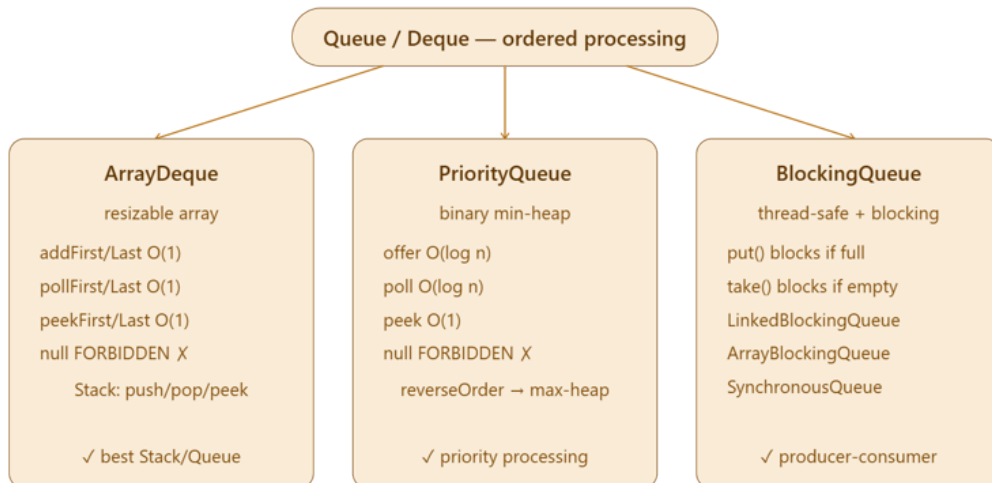


Cache locality — why ArrayList beats LinkedList in practice

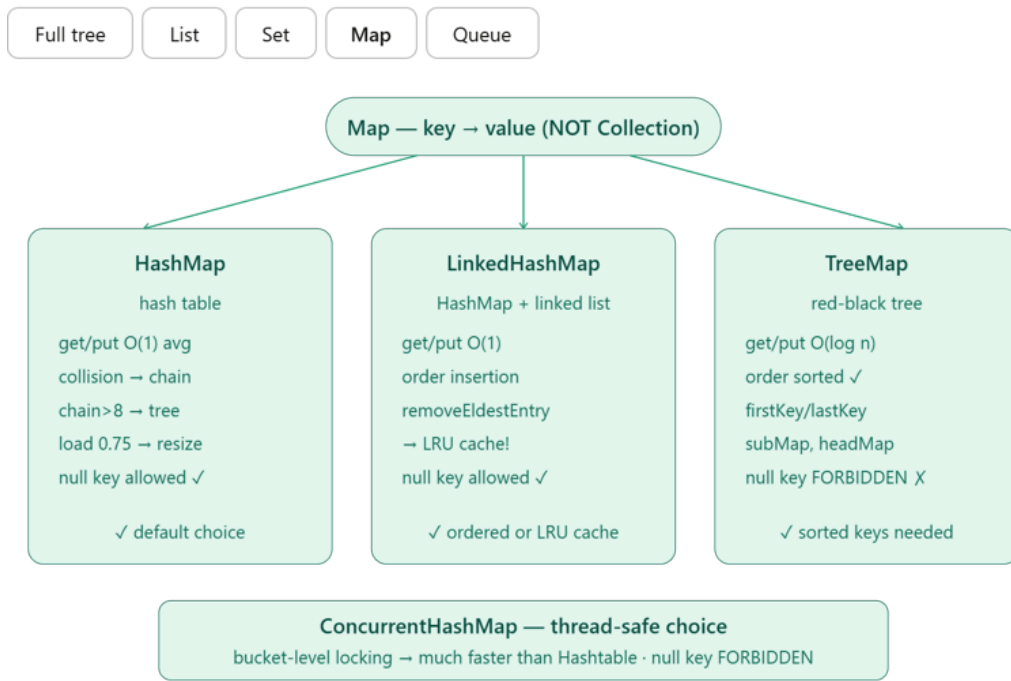
ArrayList: elements side-by-side in memory → CPU loads chunk at once → fast
 LinkedList: nodes scattered in memory → CPU must chase pointers → slow



Key rule
 HashSet and LinkedHashSet require equals() + hashCode(). TreeSet requires Comparable or Comparator.



offer/peek/poll vs add/element/remove
 offer/peek/poll → return null or false on failure (safe). add/element/remove → throw exception on failure.



Stack is in the diagram — but hidden because it's **legacy and avoided**. Let me show you exactly where it fits and why :

```

1 java.util.Stack
2     extends Vector
3         extends AbstractList
4             implements List

```

Stack is technically a List

List — ordered, index-based, duplicates OK



Other List facts

```

1 // List.of() - immutable, Java 9+
2 List<String> immutable = List.of("a", "b");
3 immutable.add("c"); // UnsupportedOperationException!
4
5 // Collections.unmodifiableList() - unmodifiable view

```

```

6 List<String> view = Collections.unmodifiableList(list);
7
8 // Sorting
9 Collections.sort(list); // natural order
10 list.sort(Comparator.comparing(String::length)); // custom

```

- Interview answer: "Use ArrayList by default. Use LinkedList only if you frequently add/remove from both ends — but even then, ArrayDeque is usually better."

What is cache locality ?

```

1 ArrayList - elements stored CONTIGUOUSLY in memory :
2 RAM: [ "Ali" | "Sara" | "Zak" | "Joe" ]
3       addr 100 101 102 103
4       ↑
5       CPU loads a "cache line" = chunk of nearby memory at once
6       → "Sara", "Zak", "Joe" already in cache → FAST
7
8 LinkedList - elements scattered RANDOMLY in memory :
9 RAM: "Ali" at addr 100 → pointer → "Sara" at addr 847
10      → pointer → "Zak" at addr 203
11      ↑
12      CPU must follow each pointer to a random address
13      → cache miss every time → much SLOWER

```

So even though LinkedList has $O(1)$ add at head theoretically — in practice ArrayList is often faster because of cache locality. The CPU doesn't have to jump around in memory.

Map — key → value, keys unique

Implementation	Order	get/put	Key requirement
HashMap	✗ none	$O(1)$	equals + hashCode
LinkedHashMap	✓ insertion order	$O(1)$	equals + hashCode
TreeMap	✓ sorted (natural)	$O(\log n)$	Comparable or Comparator

```

1 // HashMap - most common
2 Map<String, Integer> map = new HashMap<>();
3 map.put("Ali", 30);
4 map.get("Ali"); // 30
5 map.getOrDefault("X", 0); // 0 - no NPE
6 map.putIfAbsent("Ali", 99); // ignored - already exists
7 map.computeIfAbsent("B", k -> k.length()); // adds "B"→1
8
9 // Iterate entries
10 for (Map.Entry<String, Integer> e : map.entrySet()) {
11     System.out.println(e.getKey() + " → " + e.getValue());
12 }
13 // or with streams
14 map.forEach((k, v) -> System.out.println(k + " → " + v));

```

HashMap allows ONE null key and multiple null values. TreeMap does NOT allow null keys.

How HashMap works internally

```
1 // When you do: map.put("Ali", 30)
2
3 // Step 1 - compute hash of key
4 int hash = "Ali".hashCode(); // e.g. 65328
5
6 // Step 2 - find bucket index
7 int index = hash % capacity; // e.g. index = 5
8
9 // Step 3 - store in bucket (linked list)
10 // If two keys → same bucket → collision!
11 // Java chains them in a linked list
12
13 // Step 4 - Java 8+ optimization
14 // If bucket size > 8 → convert to red-black tree
15 // get/put: O(1) avg, O(log n) worst case
16
17
18 // "Ali" → bucket 5
19 // "Mia" → also bucket 5 ← collision
20 // Java chains them:
21 // bucket[5] → ["Ali", 30] → ["Mia", 25]
22 // get("Mia") → go to bucket 5
23 // → check equals("Mia") on each node
```

- ❗ If you override `equals()` without overriding `hashCode()` → two equal objects may land in different buckets → `get()` returns null even though the key exists!

HashMap uses an array of buckets. Key is hashed to find the bucket. Collisions are handled by a linked list, converted to a red-black tree after 8 entries. Load factor 0.75 triggers resize.

HashSet vs LinkedHashSet vs TreeSet

Implementation	Order	add/contains	Null
HashSet	✗ none	O(1)	✓ one null
LinkedHashSet	✓ insertion order	O(1)	✓ one null
TreeSet	✓ sorted	O(log n)	✗ no null

```
1 // HashSet - backed by HashMap internally!
2 Set<String> set = new HashSet<>();
3 set.add("Ali");
4 set.add("Ali"); // ignored - duplicate
5 set.size(); // 1
6
7 // TreeSet - sorted automatically
8 Set<Integer> sorted = new TreeSet<>();
9 sorted.add(5); sorted.add(1); sorted.add(3);
```

```

10 System.out.println(sorted); // [1, 3, 5]
11
12 // Set operations - interview classic
13 Set<String> a = new HashSet<>(Set.of("a","b","c"));
14 Set<String> b = new HashSet<>(Set.of("b","c","d"));
15 a.retainAll(b); // intersection → [b, c]
16 a.addAll(b); // union {a, b, c, d}
17 a.removeAll(b); // difference

```

HashMap stocke des paires clé → valeur.

HashSet ne stocke que des clés — pas de valeurs.

Donc comment Java implémente HashSet en utilisant HashMap ? Il invente une fausse valeur — un **dummy**

```

1 // HashSet utilise les mêmes règles que HashMap pour les clés
2 // → equals() détermine si deux éléments sont "égaux"
3 // → hashCode() détermine dans quel bucket ils vont
4
5 Set<Point> set = new HashSet<>();
6 set.add(new Point(1, 2));
7 set.add(new Point(1, 2)); // doublon ? seulement si equals() +
  hashCode() corrects!
8
9 // Si tu n'override pas equals/hashCode → deux objets différents en
  mémoire
10 // → HashSet les considère DIFFÉRENTS → set.size() = 2 ← bug!

```

Queue & Deque — Ordered processing

Queue — FIFO (first in, first out) and Deque — double-ended queue

QUEUE — FIFO (FIRST IN, FIRST OUT)

```

// Like a supermarket line
Queue<String> q =
    new LinkedList<>();
q.offer("Ali"); // add to tail
q.offer("Sara");
q.peek(); // "Ali" - see head
q.poll(); // "Ali" - remove head
// offer/peek/poll → no exception
// add/element/remove → throw exception

```

DEQUE — DOUBLE-ENDED QUEUE

```

// Stack AND Queue in one
Deque<String> dq =
    new ArrayDeque<>();
dq.addFirst("Ali"); // push front
dq.addLast("Sara"); // push back
dq.peekFirst(); // "Ali"
dq.pollLast(); // "Sara"

// Use as Stack:
dq.push("x"); // = addFirst
dq.pop(); // = removeFirst

```

PriorityQueue — sorted by priority

```

1 // Natural order - smallest first
2 PriorityQueue<Integer> pq = new PriorityQueue<>();
3 pq.offer(5); pq.offer(1); pq.offer(3);
4 pq.poll(); // 1 - always the smallest!
5 pq.poll(); // 3
6
7 // Max heap - largest first
8 PriorityQueue<Integer> maxPQ =
9     new PriorityQueue<>(Comparator.reverseOrder());
10 maxPQ.offer(5); maxPQ.offer(1); maxPQ.offer(3);

```

Complexity

Collection	get/search	add	remove	Notes
ArrayList	$O(1)$	$O(1)^*$	$O(n)$	*amortized, worst $O(n)$ resize
LinkedList	$O(n)$	$O(1)$ ends	$O(1)$ ends	$O(n)$ middle
HashMap	$O(1)$	$O(1)$	$O(1)$	avg, $O(n)$ worst collision
TreeMap	$O(\log n)$	$O(\log n)$	$O(\log n)$	always sorted
HashSet	$O(1)$	$O(1)$	$O(1)$	backed by HashMap
TreeSet	$O(\log n)$	$O(\log n)$	$O(\log n)$	backed by TreeMap
ArrayDeque	$O(n)$	$O(1)$ ends	$O(1)$ ends	best Stack/Queue
PriorityQueue	$O(\log n)$	$O(\log n)$	$O(\log n)$	peek $O(1)$

i ArrayList vs LinkedList?" → ArrayList for random access $O(1)$. LinkedList for frequent head/tail operations. In practice, ArrayList wins almost always because of cache locality.

Stack

Why never use Stack class ?

Stack is technically a List — which makes no sense for a stack data structure. That's the problem :

```

1 Stack<String> s = new Stack<>();
2 s.push("Ali");
3 s.push("Sara");
4
5 // Nonsense operations that Stack inherits from Vector/List :
6 s.get(0);           // random access on a stack?
7 s.add(0, "middle"); // insert at index?
8 s.remove(0);        // remove by index?
9 s.set(0, "hacked"); // replace by index?

```

The correct replacement :

```

1 // Use Deque as a Stack - clean, no legacy baggage
2 Deque<String> stack = new ArrayDeque<>();
3
4 stack.push("Ali"); // = addFirst
5 stack.push("Sara");
6
7 stack.peek(); // "Sara" - see top 0(1)
8 stack.pop(); // "Sara" - remove top 0(1)
9 stack.pop(); // "Ali"

```

Where it sits in the schema :

```

1 Collection
2   └─ List
3       └─ Vector (legacy, synchronized)
4           └─ Stack (legacy, extends Vector - AVOID)
5
6 vs
7
8 Collection
9   └─ Queue
10      └─ Deque
11          └─ ArrayDeque ← use this as Stack ✓

```

- ❗ "Stack extends Vector which is synchronized and exposes List methods that break the stack contract. Always use `ArrayDeque` — it implements `Deque` which has `push/pop/peek` and nothing else."

Collections & thread safety

- ❗ `ArrayList`, `HashMap`, `HashSet` are NOT thread-safe. Using them from multiple threads without synchronization → data corruption.

```

1 // Option 1 - synchronized wrappers (legacy)
2 List<String> syncList = Collections.synchronizedList(new ArrayList<>());
3 Map<String,Integer> syncMap = Collections.synchronizedMap(new
4   HashMap<>());
5 // locks the WHOLE collection - slow
6
7 // Option 2 - concurrent collections (modern, preferred)
8 import java.util.concurrent.*;
9
10 ConcurrentHashMap<String,Integer> cmap = new ConcurrentHashMap<>();
11 // locks only the BUCKET - much faster!
12 CopyOnWriteArrayList<String> cowList = new CopyOnWriteArrayList<>();
13 // copy whole array on write - great for read-heavy
14
15 BlockingQueue<String> bq = new LinkedBlockingQueue<>();
16 // blocks thread until element available - producer/consumer

```

- ❗ "ConcurrentHashMap vs Hashtable?" → `Hashtable` locks the whole map (legacy). `ConcurrentHashMap` locks per bucket → much faster. Always use `ConcurrentHashMap`.

CHOOSE THE RIGHT ONE

Order + index access → **ArrayList**

Frequent head/tail ops → **ArrayDeque**

Unique elements, fast → **HashSet**

Unique + insertion order → **LinkedHashSet**

Unique + sorted → **TreeSet**

Key-value, fast → **HashMap**

Key-value + insertion order → **LinkedHashMap**

Key-value + sorted → **TreeMap**

Priority processing → **PriorityQueue**

CLASSIC INTERVIEW QUESTIONS

ArrayList vs LinkedList → ArrayList default

HashMap internals → hash → bucket → chain/tree

HashMap vs Hashtable → Hashtable legacy + synchronized

HashSet backed by → HashMap (dummy value)

Map extends Collection? → NO! Separate hierarchy

equals+hashCode → required for HashMap/Set keys

Thread-safe Map → ConcurrentHashMap

Stack class → avoid, use ArrayDeque